

THE COMPLETE ARCHITECTURE & MASTERY GUIDE

Master Kafka.

One idea — an append-only log — explained from first intuition to the last replicated byte. Every component, every guarantee, every trade-off, drawn out and told as a story until the whole machine clicks into place.



A map of the territory

Kafka looks sprawling because it is usually taught as a pile of nouns — brokers, partitions, offsets, ISR, rebalances. This guide flips that. It teaches **one idea** (the log), then shows that every other concept is just that idea, repeated, replicated, and protected. Read it front to back and the architecture assembles itself.

Three voices, on purpose

The typography encodes *how* to read each line:

Serif is the intuition — the analogy and the story that makes a concept stick.

Sans is the precise technical explanation of how the mechanism actually works.

Mono is the machine — offsets, configs, and code you will really type.

Reading the diagrams

Colour is meaning. Every box keeps the same colour for the same kind of thing across all ~25 diagrams:

- Producer / write
- Broker / cluster
- Consumer / read
- Partition / log
- Coordination
- Leader / critical

PART I · THE BIG PICTURE

- 01 **The problem Kafka solves** — from spaghetti to a spine
- 02 **The one idea: the log** — the mental model everything hangs on
- 03 **The whole machine at a glance** — the master architecture

PART II · THE CORE ABSTRACTIONS

- 04 **Records** — the atom: key, value, time, headers
- 05 **Topics & partitions** — the unit of order and parallelism
- 06 **Offsets** — every reader's private bookmark
- 07 **Keys & partitioning** — how a record finds its lane

PART III · CLUSTER & STORAGE

- 08 **Brokers & the cluster** — the machines and the metadata
- 09 **Replication** — leaders, followers, ISR, the high watermark
- 10 **The commit log on disk** — segments, page cache, zero-copy
- 11 **Retention & compaction** — delete by time vs keep-latest
- 12 **Coordination** — ZooKeeper, the controller, and KRaft

PART IV · PRODUCERS

- 13 **Inside the producer** — serialize, partition, batch, send
- 14 **acks & durability** — how hard a write is guaranteed
- 15 **Idempotence & transactions** — exactly-once, at the source

PART V · CONSUMERS

- 16 **The poll loop** — how a consumer really runs
- 17 **Consumer groups** — scaling reads without losing order
- 18 **Rebalancing** — musical chairs, eager vs cooperative
- 19 **Offsets & lag** — committing progress, measuring debt

PART VI · GUARANTEES

- 20 **Delivery semantics** — at-most / at-least / exactly-once

PART VII · THE ECOSYSTEM

- 21 **Kafka Connect** — moving data without code
- 22 **Kafka Streams** — turning the log into computation
- 23 **Schema Registry & friends** — contracts on the wire

PART VIII · MASTERY

- 24 **The trade-off triangle** + pitfalls, tuning & cheat-sheet
- 25 **Glossary** — every term in one place

I PART ONE

The Big Picture

Before any broker or partition, there is a single idea. Get the idea, and the rest of Kafka stops being a list of parts and becomes one shape seen from different angles.

CHAPTER 01 • MOTIVATION

The problem Kafka solves

Every company eventually drowns in the same swamp: too many systems that all need to know what the others are doing. The order service needs to tell the warehouse, the fraud team, the email system, the analytics warehouse, the search index. Wire them together directly and you build an **N×M tangle** — every new system multiplies the connections, and one slow consumer can stall the producer that feeds it.

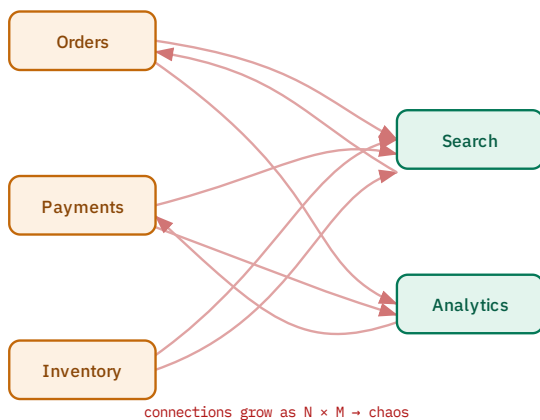
INTUITION • THE OFFICE BEFORE THE MEMO BOARD

Picture an office where every update is delivered by walking over and tapping someone on the shoulder. Sales finishes a deal and personally tells Finance, then Shipping, then Support, then the analyst in the corner. Now hire ten more teams. Suddenly people spend the whole day delivering messages by hand, and if one person is at lunch, the sender just stands there waiting.

Kafka is the moment that office installs a **memo board with a timestamp**. Sales pins one note — "deal #4182 closed" — and walks away. Anyone who cares reads the board *on their own schedule*, at their own speed. The sender never waits, the readers never miss anything, and the board keeps a permanent, ordered history of everything that ever happened. That board is the Kafka log.

Point-to-point • N×M integrations

every system wired to every other system



Log-centered • one spine, many readers

publish once • subscribe independently

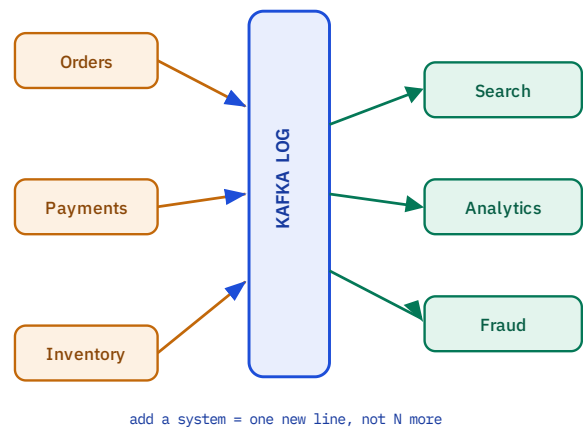


FIG 1.1 The same six systems, wired two ways. On the left, every producer must know every consumer and waits on the slowest. On the right, producers publish to one log and consumers read it independently — Kafka *decouples* who writes from who reads, in both time and space.

What Kafka actually is

Stripped to its essence, Apache Kafka is a **distributed, append-only commit log** that many producers can write to and many consumers can read from, with the whole thing replicated across machines so it survives failure. Everything else — topics, partitions, consumer groups, replication, exactly-once — is machinery built to make that one log *fast, ordered, durable, and parallel*.

Kafka gives you four super-powers that point-to-point messaging never could:

Super-power	What it means	Why it matters
Decoupling	Producers don't know who reads; consumers don't know who wrote.	Teams ship independently; add a consumer without touching producers.
Replay	Data is retained, so a reader can rewind to any past offset.	Reprocess history, backfill a new service, recover from a bug.
Buffering	The log absorbs bursts; consumers catch up at their own pace.	A slow downstream system can't stall the upstream one.
Fan-out	One write is read by many independent consumer groups.	Search, analytics, and fraud all read the same stream, isolated.

KEY IDEA

Kafka is not a queue you drain and a database you query — it is a **log you append to and replay**. A queue forgets a message once it's consumed; Kafka remembers it for as long as you configure, and lets any number of readers consume it at any speed, again and again.

The one idea: the log

If you remember only one sentence from this guide, make it this: **a Kafka partition is an append-only, totally-ordered, immutable sequence of records, and a position in that sequence is called an offset.** Master that one shape and every other concept becomes obvious.

INTUITION • THE BANK LEDGER

Think of a bank's ledger — not your balance, but the **list of every transaction that ever happened**, written in ink, one line per event, in the exact order they occurred. You never erase a line. You never insert a line in the middle. You only ever add the next line at the bottom. Your current balance is just what you get by replaying every line from the top.

That is a Kafka log. The records are the transactions. The "bottom of the page" is the **head**, where new records are appended. The line number is the **offset**. And because the ledger is never rewritten, ten different accountants can each read it at their own pace, keep their own bookmark, and always agree on what happened and in what order. Immutability is what makes the whole system trustworthy.



FIG 1.2 The anatomy of a single partition. Writes only ever happen at the head (right). Records are immutable once written. Two readers sit at different offsets and move at their own speed — the heart of why Kafka scales reads so cleanly.

Why every property of Kafka falls out of "the log"

Because the log is...	...you get this Kafka behaviour
Ordered	Records in a partition are read in exactly the order written — Kafka's core ordering guarantee.
Append-only & immutable	Sequential disk writes (extremely fast) and safe concurrent reads with no locking.
Offset-addressed	Each consumer just remembers a number; rewinding is "set the number lower"; many readers are trivial.
Retained, not deleted-on-read	Replay, fan-out to many consumer groups, and reprocessing all become possible.

A COMMON CONFUSION, SETTLED EARLY

There are **two different offsets** people mean by the word. The *log-end offset* is a property of the partition (how many records exist). The *consumer's committed offset* is a bookmark owned by a consumer group (how far it has read). They are different numbers stored in different places. Chapter 6 untangles all four positions on a partition.

Everything, on one page

Here is the entire Kafka architecture in a single diagram — producers writing in, a cluster of brokers storing replicated partitions, a control plane electing leaders, and consumer groups reading out. Every later chapter is a zoom-in on one region of this picture. Keep a finger here.

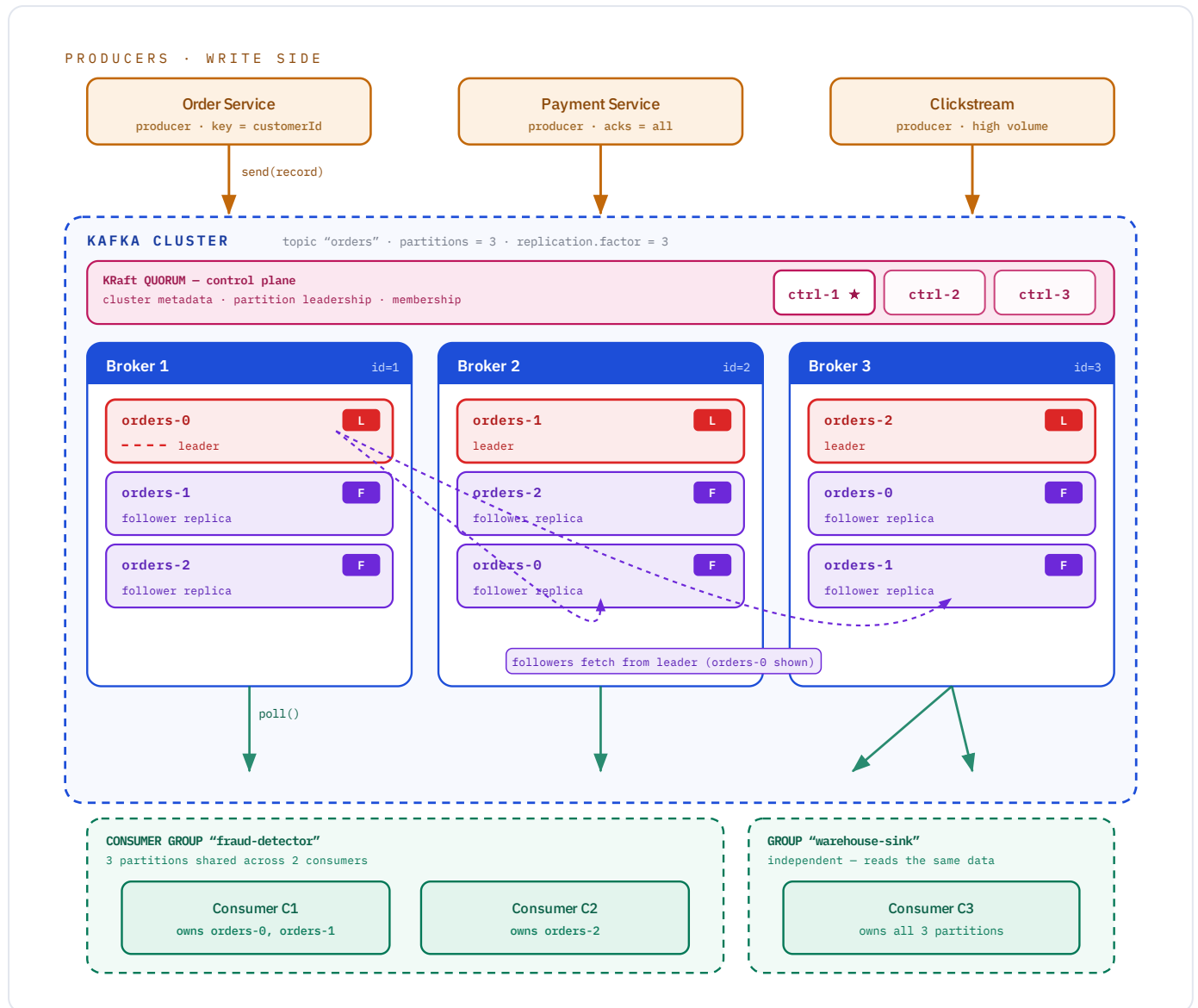


FIG 1.3 • THE MASTER DIAGRAM Producers append records (top). The cluster stores the topic orders as 3 partitions, each replicated 3× and spread across brokers so every broker leads one partition and follows two. The KRaft control plane elects leaders and tracks metadata. Two consumer groups read independently; within a group, each partition is owned by exactly one consumer.

producer (orange box) broker (blue box) partition leader (L) (red box) follower replica (F) (purple box) KRaft controller (pink box) consumer group (green box)

II

PART TWO

The Core Abstractions

Four nouns carry the whole model: the record, the topic, the partition, and the offset. Everything else in Kafka is built to store, route, replicate, and track these four.

CHAPTER 04 • THE ATOM

Records — the unit of everything

A **record** (often called a message or event) is the smallest thing Kafka stores. It is mostly just a **key** and a **value** — both raw bytes — wrapped in a little metadata. Understand the record's five fields and you understand what producers send, what brokers store, and what consumers receive.

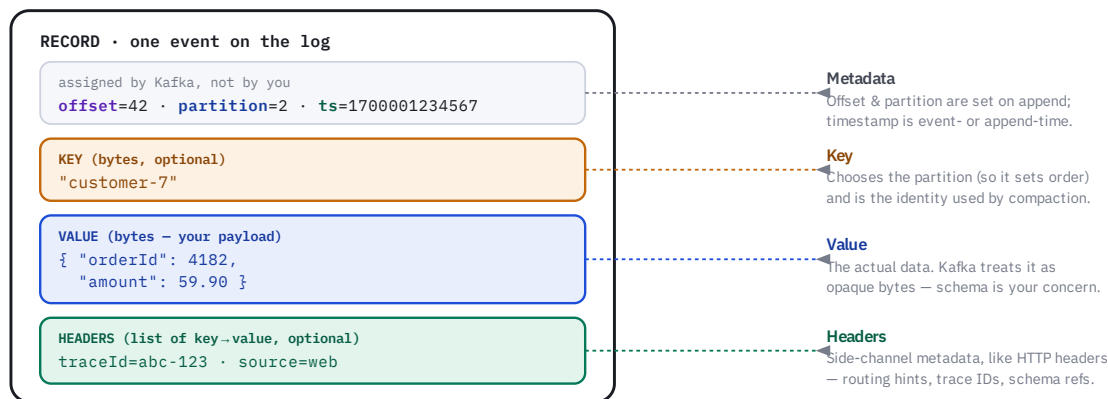


FIG 2.1 The five parts of a record. You supply key, value, and (optionally) headers; Kafka stamps the offset and partition on append. Crucially, key and value are *just bytes* to Kafka — turning your objects into bytes and back is the job of serializers (Ch 13).

INTUITION • A POSTCARD

A record is a postcard. The **value** is the message you wrote. The **key** is the recipient's surname — the post office uses it to decide which sorting bin (partition) the card drops into, so every card for the same family lands in the same bin, in order. The **headers** are the little notes in the margin ("fragile", "par avion"). And the postmark — the **offset** and **timestamp** — is stamped *by the post office* when the card is accepted, not by you.

GOTCHA • THE KEY DOES DOUBLE DUTY

A record key is **not** a database primary key and it is **not** required to be unique. Its two real jobs are (1) choosing the partition and (2) acting as the identity for log compaction. Many beginners leave the key `null` and then wonder why related events land on different partitions and lose their relative order. If order-per-entity matters, set a key.

Topics & partitions

A **topic** is a named stream of records — "orders", "payments", "clicks". But a topic is not *one* log; it is split into **partitions**, and each partition is an independent log of its own. Partitions are the single most important design decision in Kafka, because they are simultaneously the unit of **ordering**, the unit of **parallelism**, and the unit of **replication**.

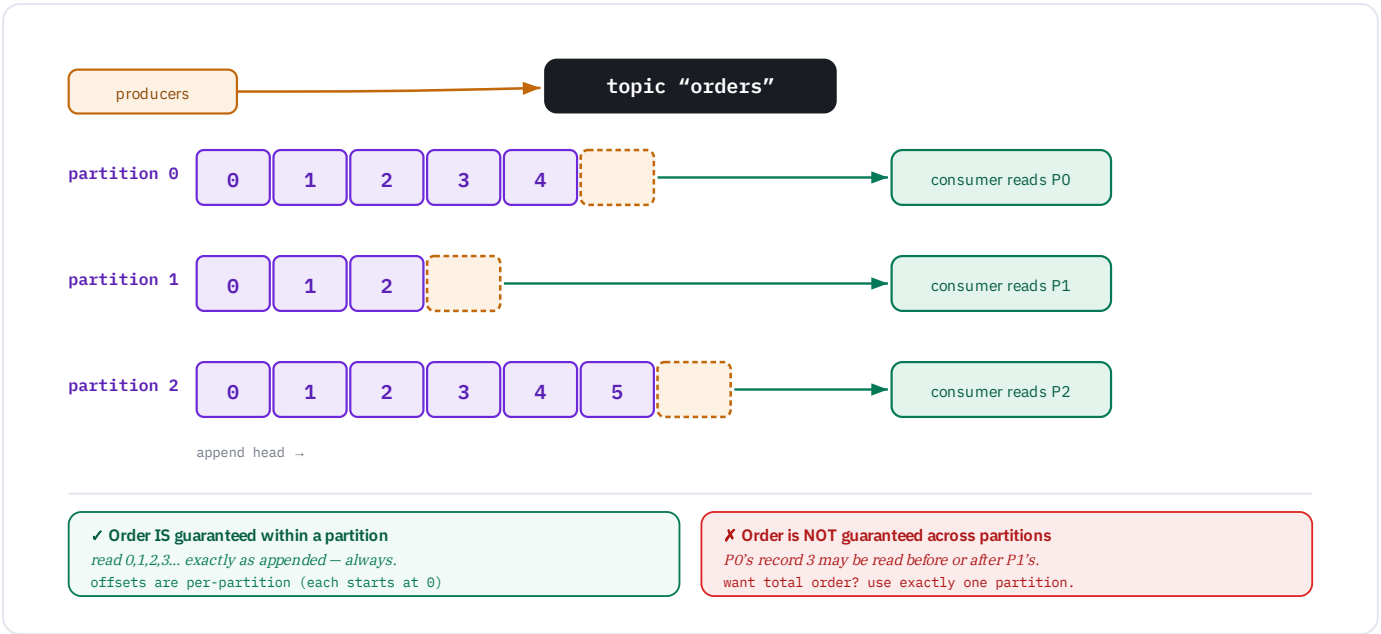


FIG 2.2 One topic, three partitions. Each partition is its own ordered log with its own offset counter (all starting at 0). More partitions means more consumers can read in parallel — but ordering only holds *inside* a partition, never across the whole topic.

Why partition count is the knob that governs scale

Throughput in Kafka is fundamentally "partitions × per-partition speed." A topic with one partition can be read by only one consumer in a group at a time; a topic with twelve partitions can be read by up to twelve. This gives the cardinal rule of capacity planning.

KEY IDEA • THE PARALLELISM CEILING

The maximum number of **actively working consumers in a single group** equals the number of **partitions**. Add more consumers than partitions and the extras sit idle. So you size partitions for your *peak future* consumer parallelism — because raising partition count later **changes how keys map to partitions** and can break per-key ordering.

More partitions...	Upside	Cost / downside
↑ parallelism	Higher throughput; more consumers can work at once.	More open files and more total replication traffic.
↑ even load	Leadership and data spread more evenly across brokers.	Longer leader-election time during failover at huge counts.
↓ batching benefit	—	Each partition gets fewer records per batch, so producer efficiency can drop.
↑ end-to-end latency	—	Very high partition counts add per-partition overhead and rebalance time.

Rule of thumb: start with a partition count a few times your expected consumer count, keep per-broker partitions in the low thousands, and prefer a key over more partitions when ordering matters.

Offsets — four numbers that explain everything

An **offset** is just an integer index into a partition. But at any moment a partition has **four** meaningful positions, and almost every consumer bug, lag alert, and "where did my message go?" question is really a confusion between two of them. Learn all four once, here.

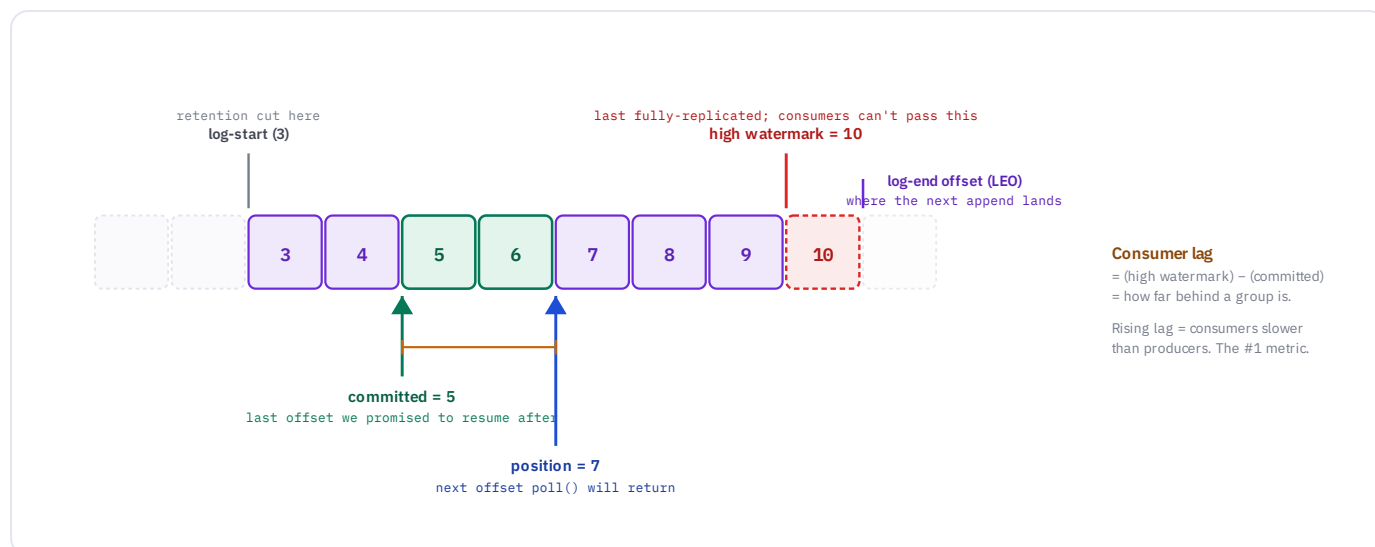


FIG 2.3 The four positions on a partition. `log-start` moves up as retention deletes old data; `committed` is the group's saved bookmark; `position` is where it's actually reading; the `high watermark` is the last replicated (and therefore readable) offset; and `LEO` is the head. The gap from `committed` to the `high watermark` is **lag**.

Position	Owned by	Meaning & why you care
log-start offset	the partition	Oldest offset still on disk. Below it, data was deleted by retention. A consumer asking for an offset below this gets an "out of range" reset.
committed offset	the consumer group	The bookmark saved in <code>__consumer_offsets</code> . On restart/rebalance, consumption resumes from here. This is "how far we've safely processed."
position	the live consumer	The in-memory pointer of the next offset to fetch. Usually a little ahead of committed (you've read but not yet committed).
high watermark	the partition leader	Highest offset replicated to all in-sync replicas. Consumers may only read <i>up to</i> here, so they never see unreplicated, possibly-lost data.
log-end offset	the partition leader	The very next offset to be written. The leader may be ahead of the high watermark while followers catch up.

GOTCHA · "I COMMITTED BUT REPROCESSED ANYWAY"

Committing offset *N* means "resume *at N* next time" — i.e., *N* is the next record to read, so everything *before N* is considered done. Off-by-one bugs here cause either **reprocessing** (committed too low) or **data skipping** (committed too high, before you actually processed). Commit *after* processing, and commit the offset of the *next* record.

Keys & partitioning — how a record finds its lane

When a producer sends a record, *something* must decide which partition it lands in. That decision — made by the **partitioner** — is what turns a record key into Kafka's most important guarantee: **all records with the same key go to the same partition, and are therefore ordered relative to one another.**

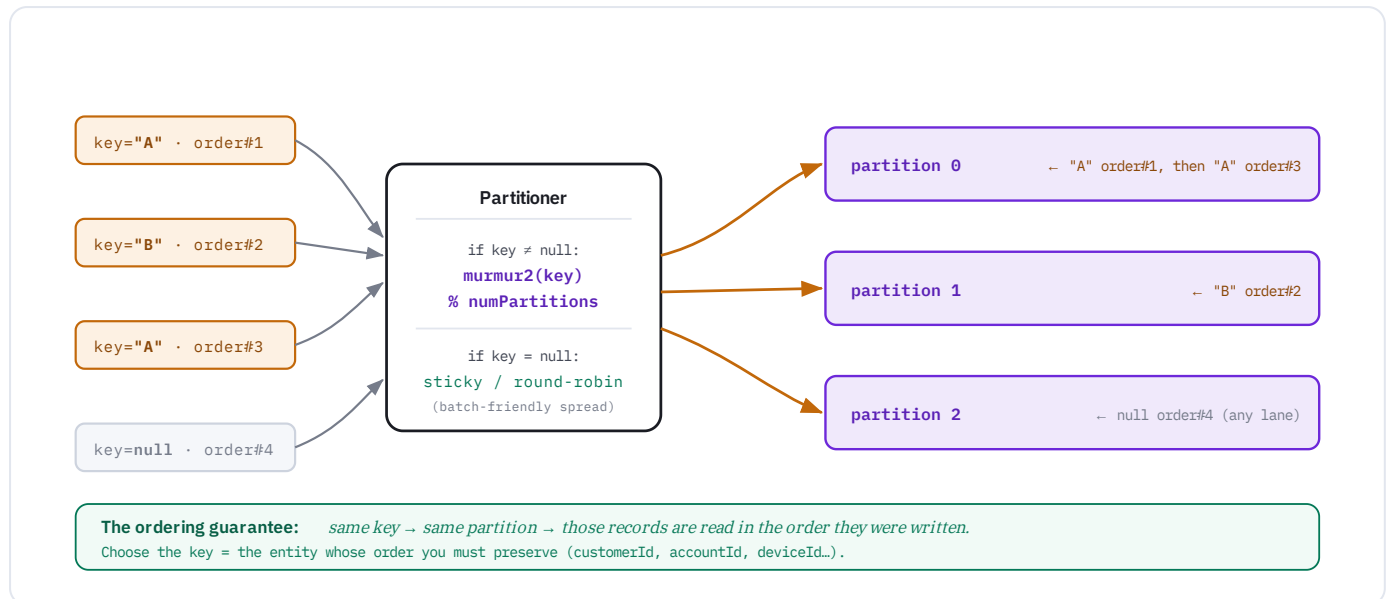


FIG 2.4 The partitioner routes each record. With a key, Kafka hashes it (`murmur2 % partitions`) so a given key is sticky to one partition — preserving per-key order. With a null key, records spread across partitions (the modern "sticky" partitioner fills one batch at a time for efficiency). You can also supply an explicit partition or a custom partitioner.

INTUITION • SUPERMARKET CHECKOUT LANES

A topic is a bank of checkout lanes; each lane is a partition. If shoppers pick any open lane (null key), you maximise throughput but a family's groceries can end up scattered across lanes and scanned out of order. If instead you route *by surname* (a key) — all the Patels to lane 3, always — then any one family is processed strictly in order, even though different families run in parallel. Kafka gives you **per-lane order with cross-lane parallelism**; the key is how you choose what "a family" means.

GOTCHA • HOT KEYS & THE RESIZING TRAP

Two real-world hazards. **(1) Key skew:** if one key is far busier than others (a celebrity userId, a giant tenant), its partition becomes a hotspot while others idle — no amount of extra consumers helps. **(2) Repartitioning:** increasing partition count changes `hash(key) % N`, so a key that mapped to partition 2 may now map to partition 5 — new records for it land elsewhere and global per-key order across the change boundary is lost. Plan partition counts up front.

The Cluster & Storage

A single log on one machine is a toy. Kafka becomes infrastructure by copying each partition across brokers, storing it as blazing-fast sequential files, and electing a brain to keep everyone in agreement.

CHAPTER 08 · THE MACHINES

Brokers & the cluster

A **broker** is a single Kafka server. It stores some partitions on its disks, serves reads and writes for the partitions it leads, and replicates the ones it follows. A **cluster** is a set of brokers that together hold all the partitions of all the topics. There is no "primary broker" — every broker is an equal that happens to lead some partitions and follow others.

How a client finds its way in: bootstrap & metadata

A producer or consumer never needs to know the whole cluster up front. You give it a few `bootstrap.servers` — any brokers will do. The client connects, asks "who leads which partition?", and receives the cluster's **metadata map**. From then on it talks *directly* to the leader of each partition it cares about, refreshing metadata when leadership changes.

INTUITION · ASKING FOR DIRECTIONS, ONCE

Bootstrap servers are like asking any person at the train station "where's the platform for the 5pm to Bristol?" You don't need a map of every platform — you just need *someone* who can tell you where to go. Once you know your platform (the partition leader), you walk straight there. If the platform changes (leader fails over), you ask again. The handful of bootstrap addresses are just "someone to ask first."

A broker is responsible for...	Detail
Leading some partitions	Handles all reads and writes for those partitions; the single source of truth for their order.
Following others	Continuously fetches from leaders to keep replica copies up to date, ready to take over.
Storing log segments	Persists records to disk as segment files and serves them from the OS page cache.
Group coordination	One broker acts as <i>group coordinator</i> for a given consumer group (manages membership & offsets).
Heartbeating to the controller	So the control plane knows it's alive; a silent broker triggers leader re-election for its partitions.

KEY IDEA · SYMMETRY IS THE SCALING TRICK

Because every broker plays the same role, you scale by simply **adding brokers**; Kafka can then rebalance partition leadership across the larger set. There is no master to become a bottleneck — load is spread by spreading *partition leadership*, which is exactly why partition count caps your parallelism.

Replication — how Kafka survives a dead machine

Each partition is copied onto several brokers. One copy is the **leader** (it takes all traffic); the rest are **followers** that pull from the leader. The followers that are sufficiently caught up form the **in-sync replica set (ISR)**. This trio — leader, followers, ISR — plus the **high watermark** is the entire durability model, and it is worth slowing down for.

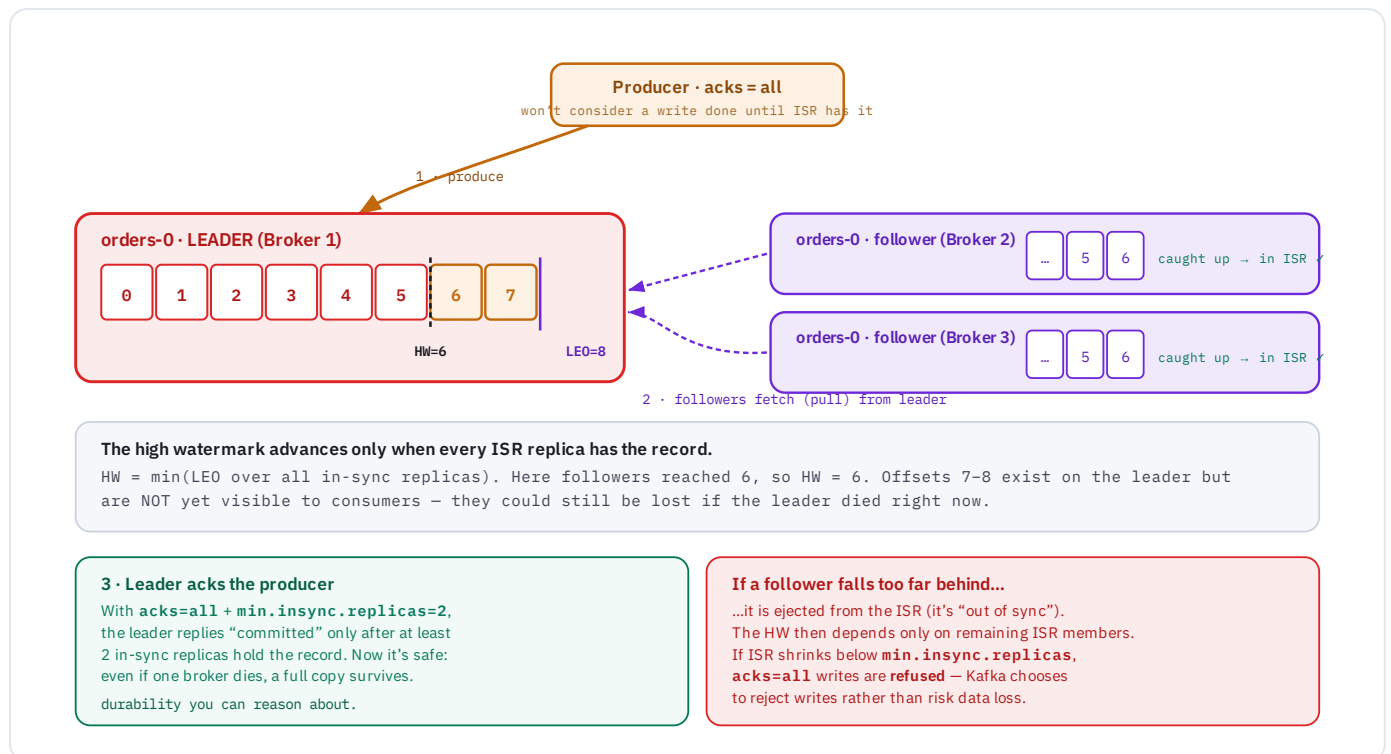


FIG 3.1 Replication in motion. The producer writes to the leader; followers pull to catch up; the high watermark — the visible, durable frontier — advances to min(LEO) across the in-sync set. With `acks=all` the producer is told "done" only once enough in-sync replicas hold the record, so a single broker failure never loses an acknowledged write.

Setting	Typical	What it controls
<code>replication.factor</code>	3	How many copies of each partition exist. RF=3 tolerates 1 broker loss with strong durability.
<code>min.insync.replicas</code>	2	Minimum ISR members required for an <code>acks=all</code> write to succeed. The durability floor.
<code>acks (producer)</code>	all	How many acknowledgements the producer waits for (see Ch 14).
<code>unclean.leader.election</code>	false	If true, an out-of-sync replica may become leader (availability over safety — risks data loss). Keep false.

KEY IDEA • THE CLASSIC SAFE TRIO

`replication.factor=3`, `min.insync.replicas=2`, and producer `acks=all`. This survives the loss of any one broker with *no* acknowledged-data loss, while still accepting writes. It is the default starting point for any data you can't afford to lose.

The commit log on disk — and why it's so fast

A partition is not one giant file; it's a directory of **segments**. Only the newest segment (the **active segment**) is open for appends — everything older is sealed and immutable. This layout, combined with two operating-system tricks, is why Kafka can push millions of messages per second on ordinary hardware.

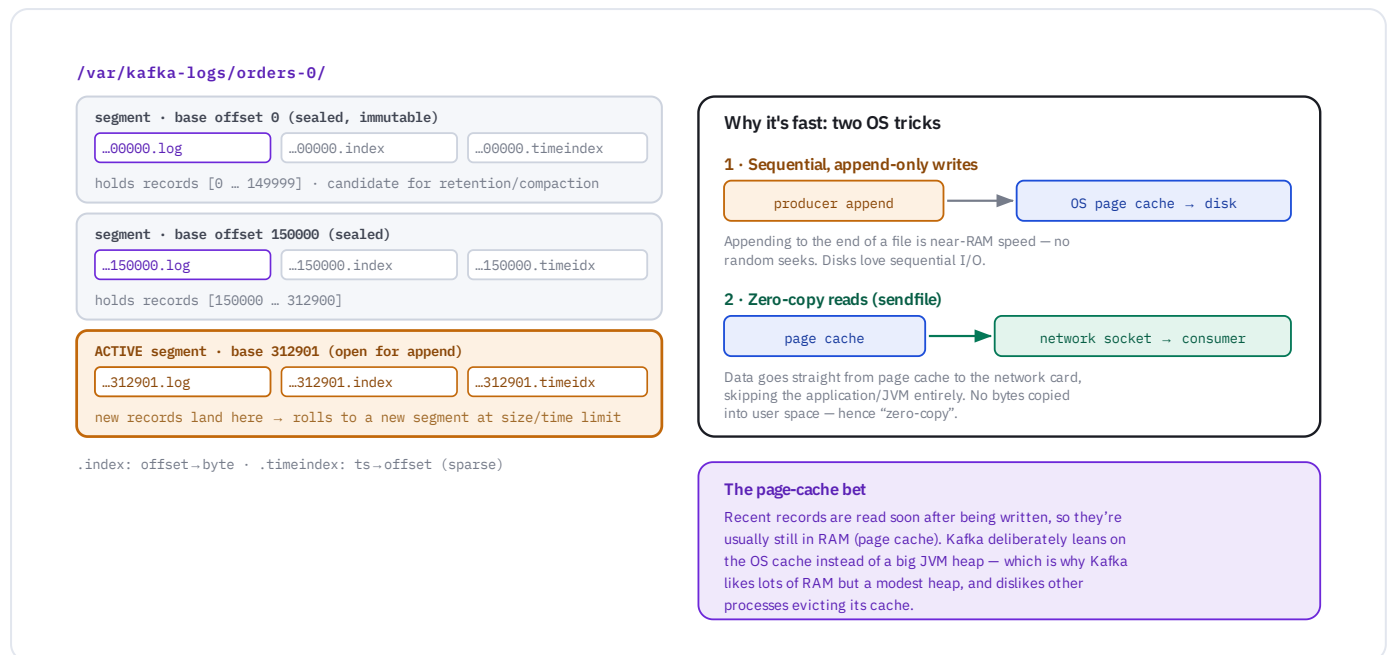


FIG 3.2 A partition on disk: a series of segment files, only the last open for writing. Sparse .index/.timeindex files let a broker jump to any offset or timestamp without scanning. Sequential writes and zero-copy reads — both gifts from the OS — are the core of Kafka's throughput.

When does a segment roll, and when is data eligible to go?

The active segment closes (and a new one opens) when it hits a size or age limit; only sealed segments can be deleted or compacted. These few knobs govern disk usage and how granular cleanup is:

Config	Default-ish	Effect
segment.bytes	1 GB	Roll to a new segment once the active one reaches this size.
segment.ms	7 days	Roll after this much time even if the size limit isn't hit.
retention.ms	7 days	Delete sealed segments whose records are older than this (delete policy).
retention.bytes	-1 (off)	Cap total partition size; oldest segments drop when exceeded.

KEY IDEA • IMMUTABILITY IS THE PERFORMANCE STORY

Because sealed segments never change, Kafka never has to do random in-place updates, locking, or page rewrites. Reads and writes are both *linear*. The price is that you “edit” data only by appending a new record (and, for tables, letting compaction collapse old versions) — never by mutating in place.

Retention & compaction — two ways to forget

Kafka keeps data, but not forever. There are two distinct cleanup policies, and choosing the right one defines what a topic *is*.

Deletion drops whole old segments by age or size — perfect for event streams. **Compaction** keeps only the latest value per key — perfect for changelogs and snapshots.

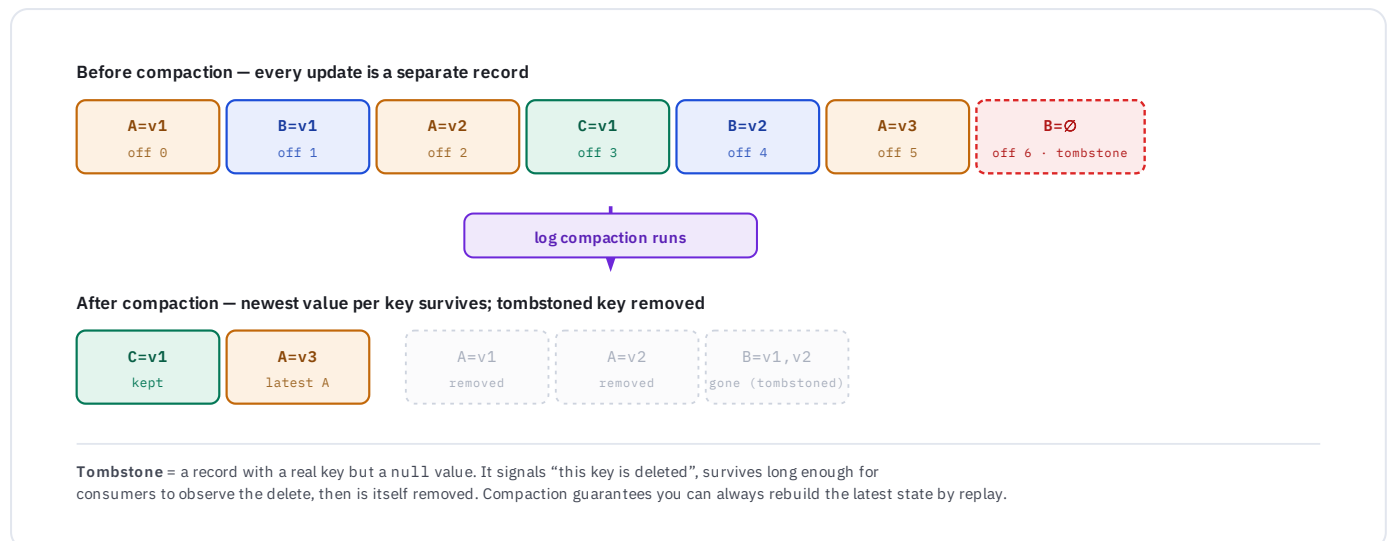


FIG 3.3 Log compaction collapses a key's history to its latest value, and a null-valued *tombstone* deletes a key entirely. The result is a compact “latest snapshot” you can replay to reconstruct current state — the mechanism behind Kafka Streams' state stores and `__consumer_offsets`.

● Delete retention (the default)

- `cleanup.policy=delete`
- Drops whole segments older than `retention.ms` or beyond `retention.bytes`.
- Models an **event stream** — “keep 7 days of orders.”
- Offsets you've passed are simply aged out; replay is bounded by the window.

● Compact retention

- `cleanup.policy=compact`
- Keeps the **latest value per key** indefinitely; removes superseded versions.
- Models a **changelog / table** — “current profile for every user.”
- You can also combine: `compact, delete`.

INTUITION • A DIARY VS AN ADDRESS BOOK

A **delete** topic is a diary: you write every day's entry and tear out pages older than a month. You keep recent history, but not forever. A **compact** topic is an address book: when a friend moves, you cross out the old address and write the new one on the same line — you don't keep a record of every address they ever had, only where they live *now*. Same log machinery, opposite relationship with the past.

Coordination — from ZooKeeper to KRaft

Someone has to remember the cluster's truth: which topics exist, which broker leads each partition, who's alive. Historically Kafka delegated this to a separate **ZooKeeper** ensemble. Modern Kafka replaces it with **KRaft** — a built-in quorum of controllers that stores metadata as, fittingly, a *Kafka log*. Same job, one fewer system to run.

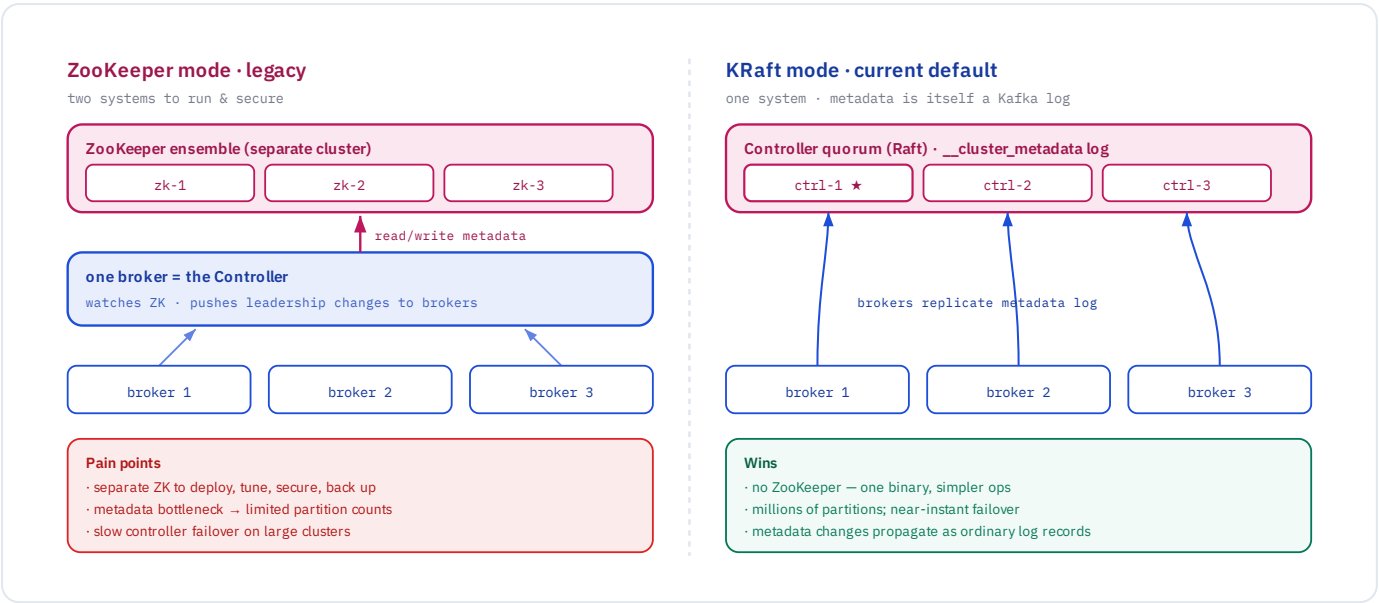


FIG 3.4 Two eras of coordination. ZooKeeper kept metadata in an external ensemble that a single elected broker-controller mirrored. KRaft folds that role into a Raft-based controller quorum whose metadata *is* a replicated Kafka log that brokers subscribe to — fewer moving parts, faster failover, and far higher partition scalability.

WHERE THINGS STAND

KRaft is production-ready and the default for new clusters; ZooKeeper mode is being retired. The controller's job is the same in both: maintain cluster metadata, detect broker failures, and orchestrate **leader election** for partitions whose leader has died. What changed is *where* that truth lives — and that it now lives in the very log abstraction this whole guide is about.

IV
PART FOUR

Producers

The write side. A producer is far more than a network call — it serializes, routes, batches, and retries, and a few of its settings decide whether your data is fast, safe, or exactly-once.

CHAPTER 13 • THE WRITE PIPELINE

Inside the producer

Calling `producer.send()` does not immediately hit the network. The record flows through a pipeline split across **two threads**: your application thread serializes and partitions it into an in-memory buffer, and a background **sender thread** batches buffered records and ships them to brokers. Understanding this hand-off explains batching, latency, back-pressure, and ordering.

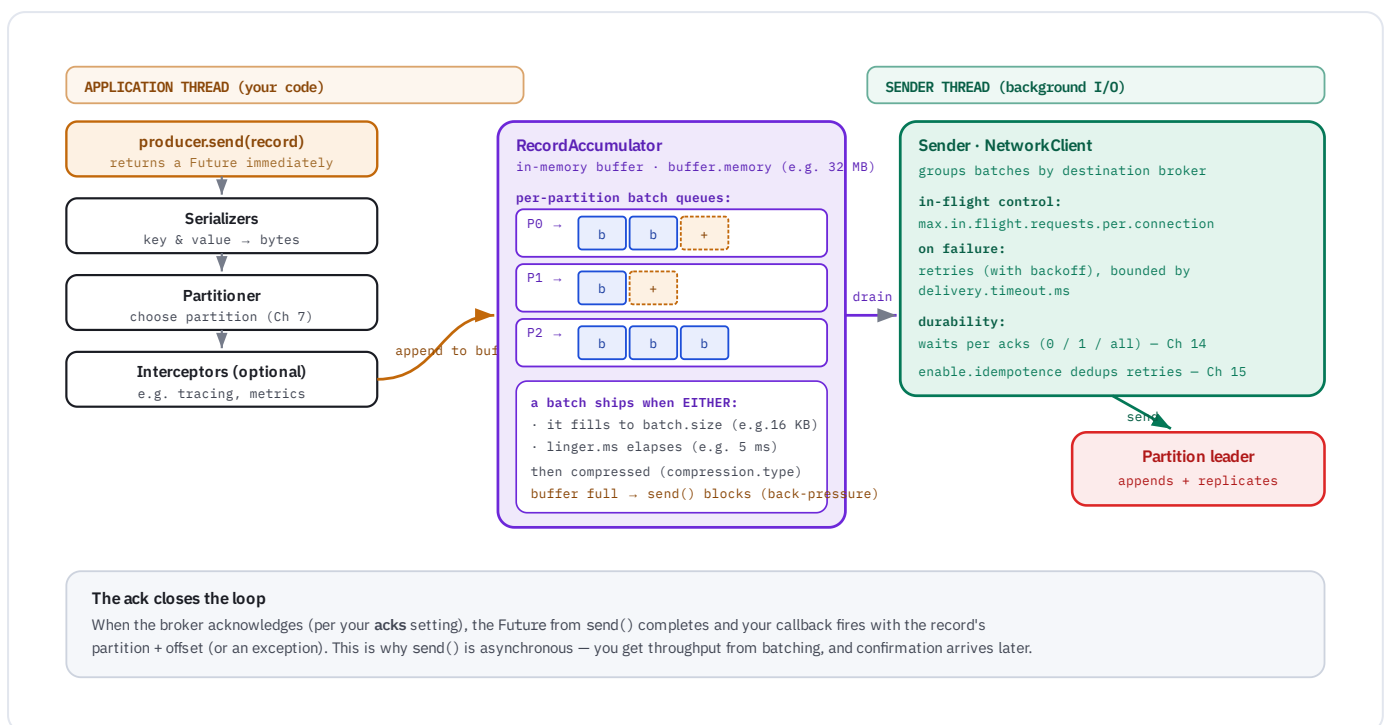


FIG 4.1 The producer pipeline. Your thread serializes, partitions, and drops records into the `RecordAccumulator`, where they batch per partition until `batch.size` or `linger.ms` triggers a `send`. The sender thread groups batches by broker and ships them, handling retries and acks. The original `send()` already returned a `Future`; its completion is the ack.

Knob	Trades	Effect
<code>batch.size</code> / <code>linger.ms</code>	latency ↔ throughput	Bigger batches / longer <code>linger</code> = fewer, larger requests = more throughput, slightly more latency.
<code>compression.type</code>	CPU ↔ network	<code>lz4/zstd/snappy/gzip</code> shrink batches; compress once, store & transfer compressed.
<code>buffer.memory</code>	memory ↔ blocking	Total buffer before <code>send()</code> blocks (back-pressure) waiting for the sender to drain.
<code>max.in.flight...</code>	throughput ↔ ordering	Unacked requests per connection. With idempotence on, up to 5 still preserves order on retry.

acks — how hard a write is guaranteed

The single most consequential producer setting is `acks`. It decides *when* the broker tells the producer "got it" — and therefore how much you're willing to risk losing for speed. There are exactly three settings, and they map cleanly onto the replication model from Chapter 9.

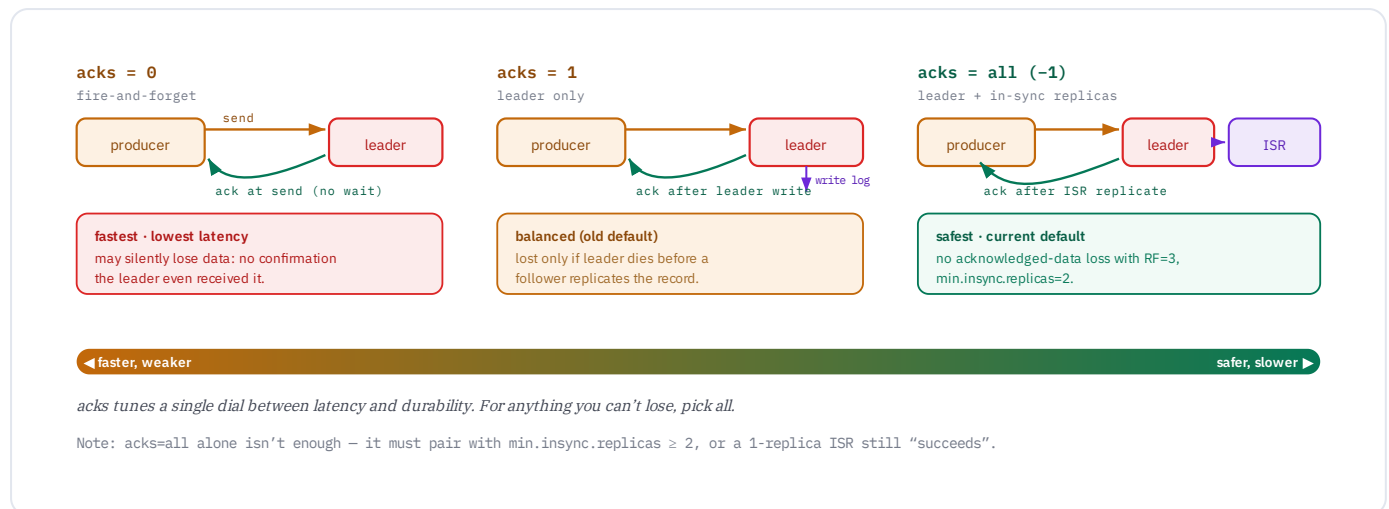


FIG 4.2 The three acknowledgement levels. `acks=0` never waits (can lose data). `acks=1` waits for the leader's local write (can lose on failover). `acks=all` waits for the in-sync replicas, and — paired with `min.insync.replicas=2` — loses no acknowledged write to a single broker failure.

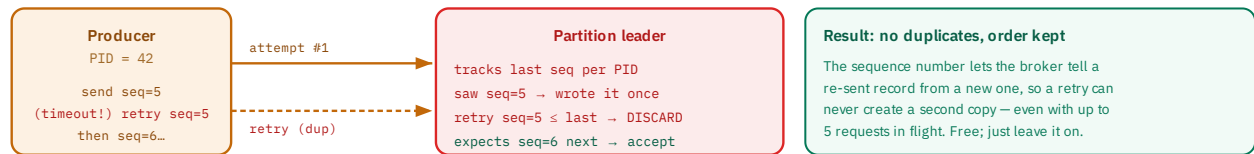
acks	Waits for	Data-loss window	Use when
0	nothing	Large — no delivery guarantee	High-volume metrics/logs where occasional loss is fine.
1	leader write	Leader fails pre-replication	Throughput-sensitive data tolerant of rare loss.
all	ISR replication	None (with <code>min.insync.replicas ≥ 2</code>)	Orders, payments, anything you must not lose. Default.

Idempotence & transactions

Retries are how producers survive flaky networks — but a naive retry can write the same record twice. Kafka solves this in two layers: the **idempotent producer** removes duplicates from retries to one partition, and **transactions** make a set of writes across many partitions all commit or all abort, atomically. Together they are the foundation of exactly-once processing.

Layer 1 · Idempotent producer — kills duplicate retries to a partition

`enable.idempotence=true` (default) · producer gets a PID; each record carries (PID, partition, sequence#)



Idempotence is per-producer-session, per-partition. It stops retry duplicates — it does not, by itself, make a multi-step pipeline exactly-once. For that, you need transactions ↴

FIG 4.3a Idempotent producer. Each record carries a producer ID and a monotonic sequence number per partition; the leader remembers the last sequence it accepted and silently drops any retry that repeats one. Duplicates from retries simply cannot reach the log.

Layer 2 · Transactions — all-or-nothing across many partitions

`transactional.id` set · `initTransactions` → `beginTransaction` → `send(...)` → `commitTransaction` / `abortTransaction`

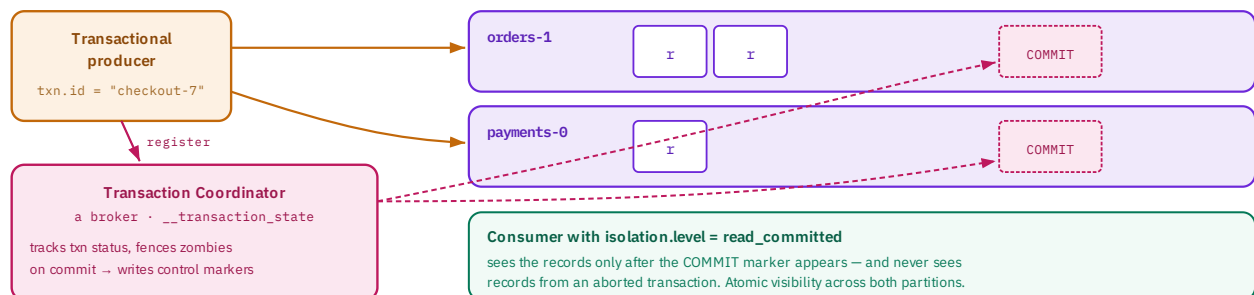


FIG 4.3b Transactions. A transactional producer registers with a coordinator (backed by `__transaction_state`), writes to multiple partitions, then commits. The coordinator stamps *control markers* into each partition; `read_committed` consumers reveal the records only once committed — so a multi-partition write is all-or-nothing.

KEY IDEA · THE EXACTLY-ONCE STACK

Idempotence stops *retry* duplicates; transactions give *atomic* multi-partition writes and fence out "zombie" producers after failover. The headline use case is the **read-process-write** loop — consume, transform, produce, and commit the consumed offsets *inside the same transaction* — which yields true end-to-end exactly-once (Ch 20 and Ch 22).

V

PART FIVE

Consumers

The read side. Consumers pull at their own pace, band together into groups to share work, and bookmark their progress with offsets. The magic — and most of the operational drama — lives in groups and rebalancing.

CHAPTER 16 · THE HEARTBEAT OF A CONSUMER

The poll loop

A consumer is fundamentally a loop around one method: `poll()`. Each call fetches a batch of records from the partitions assigned to it, you process them, and you loop again. But `poll()` does double duty — it also drives **group membership** in the background. Miss your polling deadline and the group assumes you're dead.

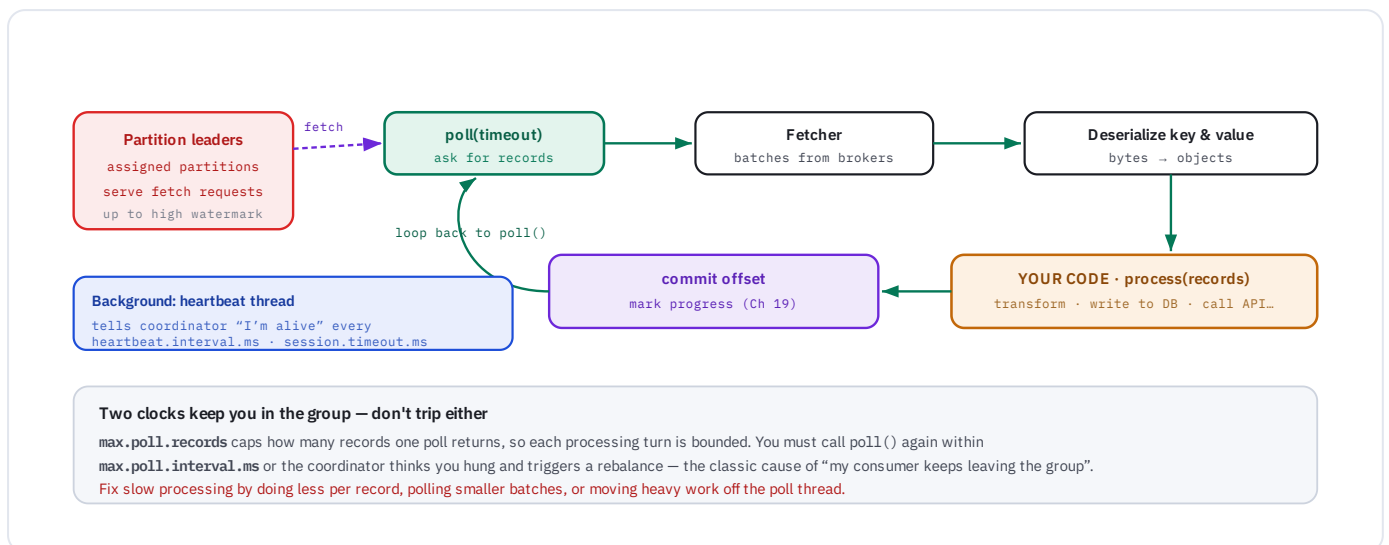


FIG 5.1 The poll loop. Each `poll()` fetches a bounded batch, you process it, then you loop. A separate heartbeat thread keeps your group membership alive, but you still must return to `poll()` within `max.poll.interval.ms` — long processing without polling looks identical to a crash.

INTUITION · THE CONVEYOR BELT

Think of a worker at a conveyor belt who grabs a small tray of items (`max.poll.records`), processes them, and reaches back for the next tray. The supervisor (group coordinator) walks by periodically; as long as the worker keeps reaching back for trays on schedule, they keep their spot on the line. Disappear into one enormous tray for too long and the supervisor assumes you've collapsed — and gives your section of the belt to someone else.

Consumer groups

A **consumer group** is how Kafka shares a topic's partitions across many consumer instances so they can process in parallel. The defining rule is beautifully simple: **within a group, each partition is owned by exactly one consumer** — and across *different* groups, everyone reads the same data independently. This one rule gives you both scaling and broadcast.

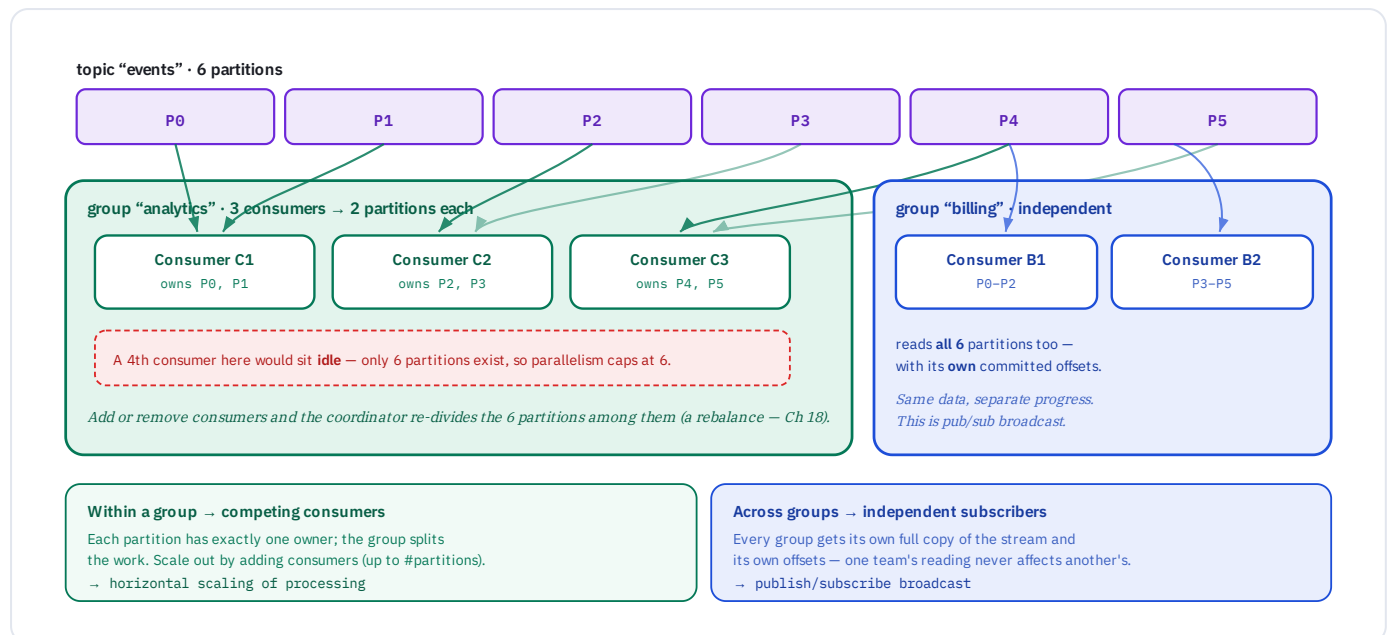


FIG 5.2 Groups do two jobs at once. *Inside* the "analytics" group, six partitions are split across three consumers (one owner each) — that's scaling. The separate "billing" group reads the very same partitions with its own offsets — that's broadcast. The `group.id` is what places a consumer into a group.

GOTCHA • IDLE CONSUMERS & THE PARTITION CEILING

Because a partition has exactly one owner per group, **you can never have more working consumers in a group than partitions**. Spin up eight consumers on a six-partition topic and two will sit idle, contributing nothing. This is the read-side echo of the same ceiling from Chapter 5: partition count is your unit of parallelism. Size partitions for your peak consumer count.

Rebalancing — when partitions change hands

When a consumer joins, leaves, or dies — or partitions are added — the group must re-divide partitions among its members. This is a **rebalance**, run by the group coordinator. Rebalances are necessary, but the *old* way pauses everyone; modern Kafka makes them incremental so most consumers keep working. Knowing the difference is the key to a stable consumer fleet.

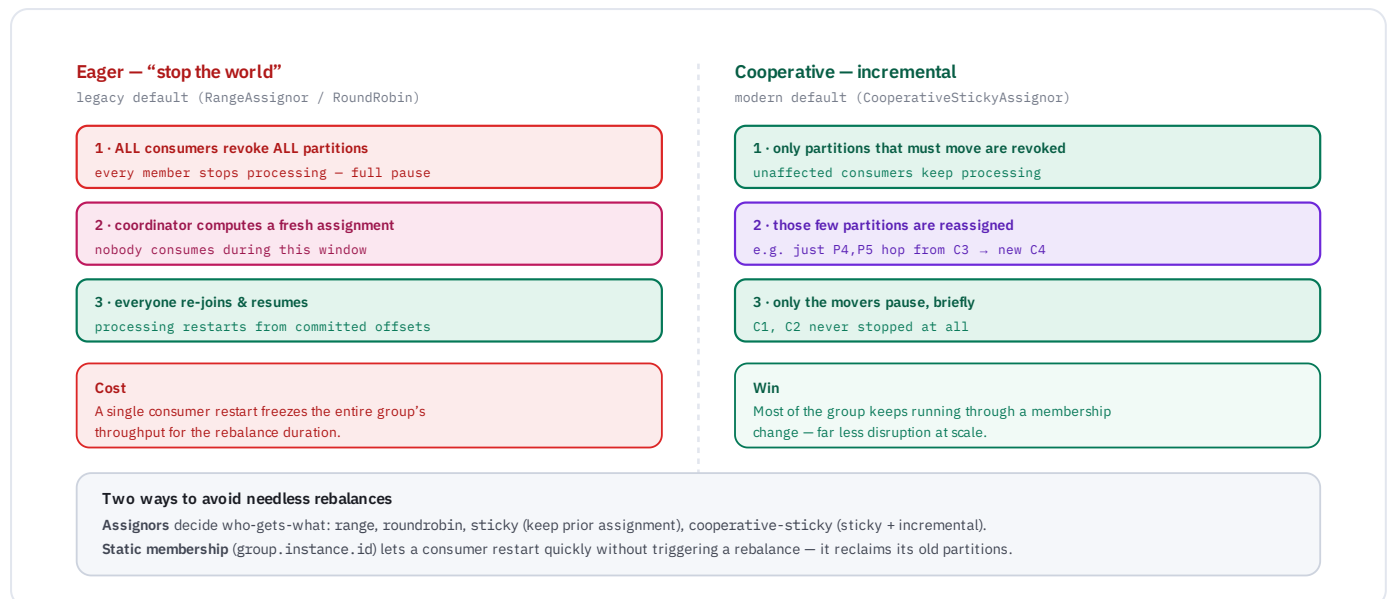


FIG 5.3 Eager vs cooperative rebalancing. Eager revokes everything and pauses the whole group; cooperative revokes only the partitions that actually need to move, so unaffected consumers never stop. Combined with sticky assignors and static membership, rebalances become cheap instead of catastrophic.

INTUITION • MUSICAL CHAIRS VS SWAPPING ONE SEAT

Eager rebalancing is musical chairs: the music stops, *everyone* leaps up, and all players scramble for seats again — chaos even if only one player joined. Cooperative rebalancing is the polite version: a newcomer walks in, two people quietly slide over to free one chair for them, and everyone else keeps sitting and chatting. Same end state, a fraction of the disruption.

Offsets, commits & lag

A group remembers how far it has read by **committing offsets** — saved into an internal compacted topic, `__consumer_offsets`. *When* you commit, relative to when you process, decides your delivery guarantee. And the gap between what's been produced and what you've committed is **lag**: the single most important number to watch in production.

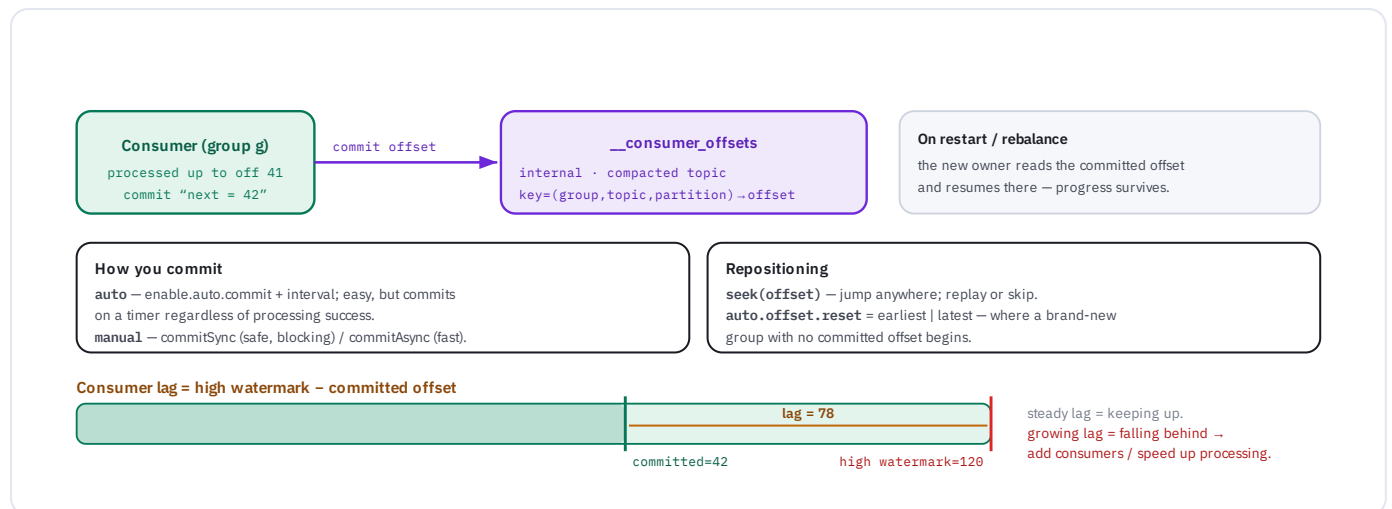


FIG 5.4 Offsets are committed to the compacted `__consumer_offsets` topic, so progress survives restarts and rebalances. You commit automatically (timer) or manually (`commitSync`/`commitAsync`), and can seek to replay. Lag — high watermark minus committed — tells you whether consumers are keeping up.

GOTCHA • COMMIT TIMING IS YOUR DELIVERY GUARANTEE

If you commit *after* successfully processing, a crash mid-batch means you'll reprocess the uncommitted records — **at-least-once** (possible duplicates, never loss). If you commit *before* processing (or rely on the auto-commit timer firing between fetch and processing), a crash means those records are marked done but never handled — **at-most-once** (possible loss, never duplicates). There is no third "just works" option here — you choose by *where* you place the commit. This is the doorway to Part VI.

VI

PART SIX

Delivery Guarantees

Everything so far — acks, replication, commits, transactions — converges here, on the question people actually ask: "will I lose messages, or see them twice?" The answer is a choice, and now you can make it deliberately.

CHAPTER 20 • THE THREE SEMANTICS

At-most-once, at-least-once, exactly-once

There are exactly three possible delivery guarantees, and they differ in one tiny decision: the **order of "process" and "commit."** Get this ordering and its failure modes straight and you'll never again be surprised by a lost or duplicated message.

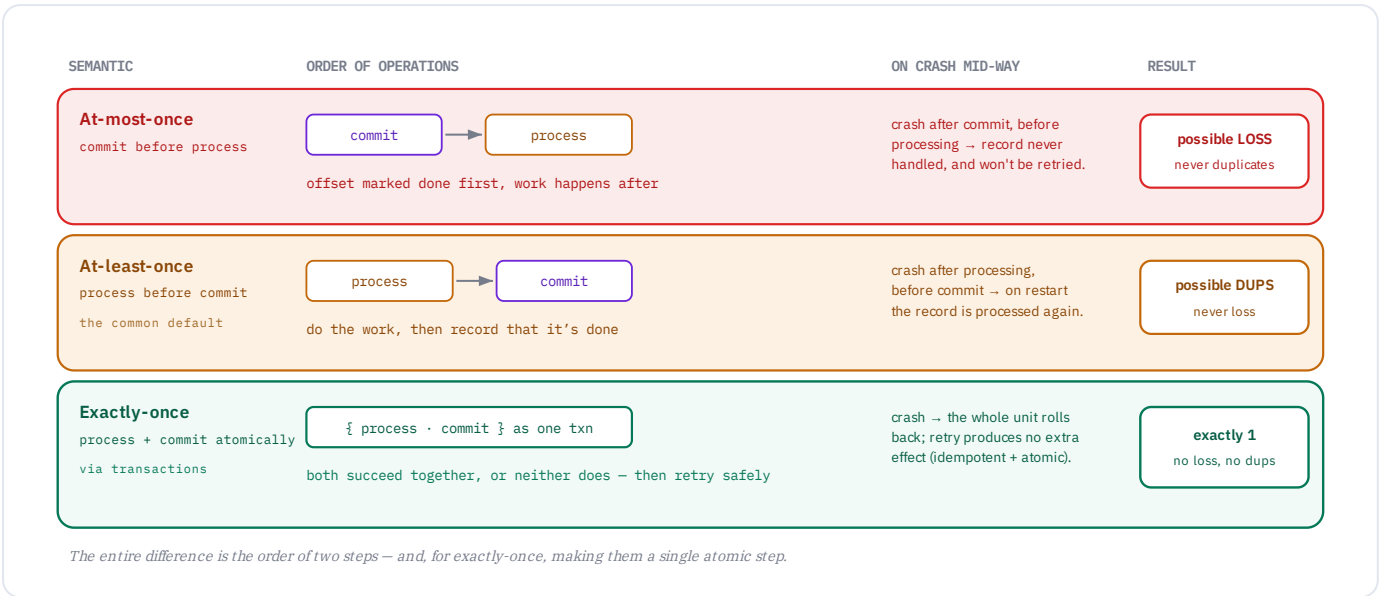


FIG 6.1 Three guarantees, one decision. Commit-then-process risks loss (at-most-once); process-then-commit risks duplicates (at-least-once); doing both atomically in a transaction yields exactly-once. Most systems run at-least-once and make processing *idempotent* downstream; true exactly-once needs Kafka transactions.

KEY IDEA • "EXACTLY-ONCE" MEANS EXACTLY-ONCE EFFECT

Kafka can't stop a record from being physically delivered twice across a network — what it guarantees is that the **observable effect** happens once. It does this two ways: idempotence (a retried write is recognized and dropped) and transactions (a unit of work commits all-or-nothing). "Exactly-once processing" is the combination applied to a consume-process-produce pipeline.

End-to-end exactly-once: the read-process-write loop

The headline pattern is a stage that *reads* from one topic, *transforms*, and *writes* to another. The trick that makes it exactly-once is committing the **input offsets inside the same transaction as the output records** — so "I produced the result" and "I consumed the input" are one atomic fact.

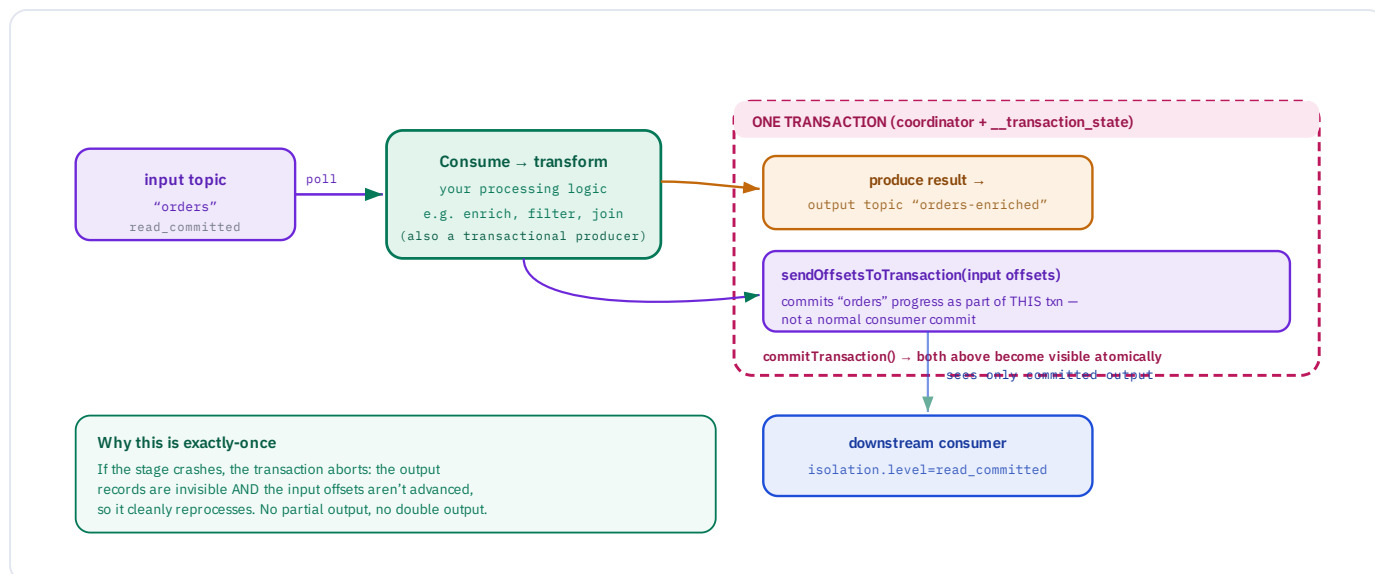


FIG 6.2 End-to-end exactly-once. The processor is a transactional producer: it writes its output *and* commits the input offsets inside a single transaction via `sendOffsetsToTransaction`. Either both land or neither does, so a crash reprocesses cleanly. Downstream consumers read with `read_committed` to see only finished work.

GOTCHA • EXACTLY-ONCE HAS A SCOPE AND A COST

Kafka's exactly-once covers the **Kafka-to-Kafka** path (and Kafka Streams builds on it with one config). The moment your processing also writes to an *external* system — a database, an email, a payment API — that side effect is outside Kafka's transaction, so you need idempotent writes or an outbox pattern there. Transactions also add latency and throughput overhead; reach for at-least-once + idempotent handlers unless you truly need EOS.

VII

PART SEVEN

The Ecosystem

Raw producers and consumers are the assembly language of Kafka. Three official tools sit on top: Connect moves data in and out without code, Streams processes it as a library, and the Schema Registry keeps everyone speaking the same format.

CHAPTER 21 · INTEGRATION WITHOUT CODE

Kafka Connect

Kafka Connect is a framework for streaming data between Kafka and other systems using only *configuration* — no bespoke producer/consumer code. **Source** connectors pull from external systems into Kafka; **sink** connectors push from Kafka out to them. It runs as its own scalable, fault-tolerant cluster of workers, and there's a large library of ready-made connectors.



FIG 7.1 Kafka Connect. Source connectors ingest from external systems into Kafka; sink connectors export the other way — all via configuration. A distributed worker cluster runs the connectors' tasks, rebalancing them on failure, and stores its own state in internal topics. Converters and SMTs handle serialization and light per-message transforms.

Kafka Streams

Kafka Streams is a Java library — not a separate cluster — for transforming and aggregating Kafka topics in your own application. You declare a **topology** (a graph of processing steps); it handles partitioning, parallelism, fault-tolerant local **state**, and exactly-once for you. Its central idea is the duality between a **stream** and a **table**.

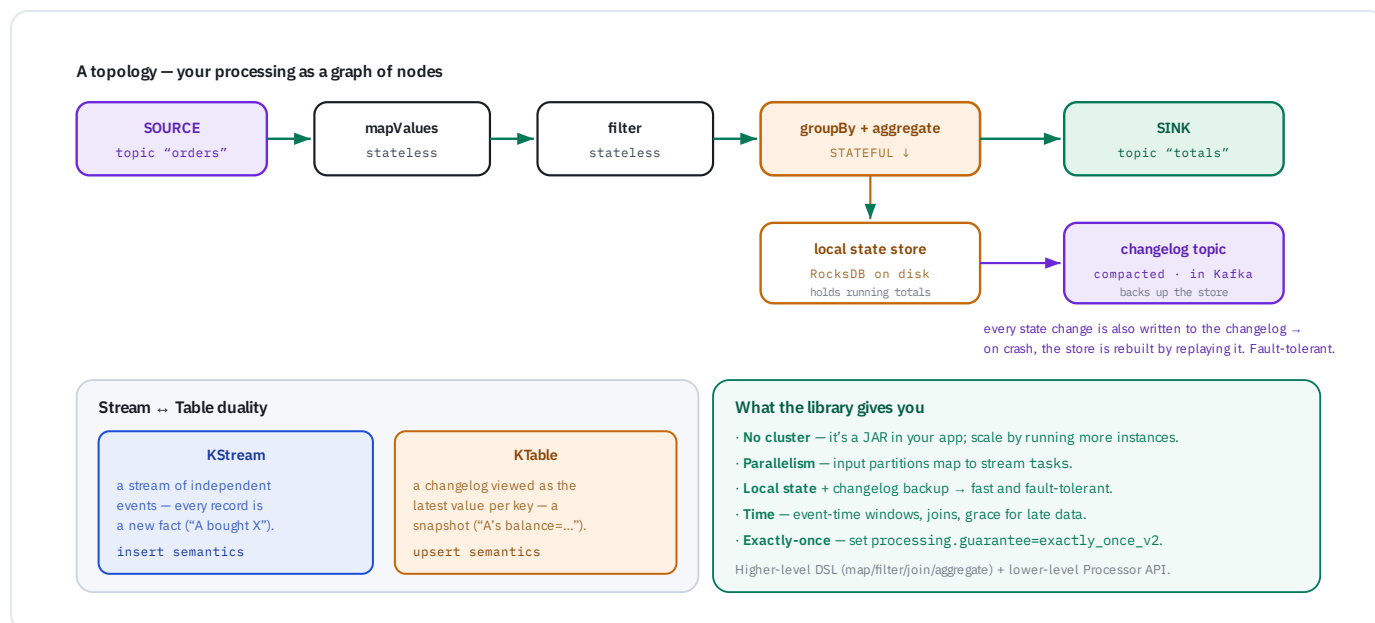


FIG 7.2 Kafka Streams. You build a topology of stateless and stateful nodes; stateful ones keep a fast local store (RocksDB) backed by a compacted *changelog* topic for recovery. The stream/table duality — events versus latest-value-per-key — underpins joins and aggregations, and the whole thing runs as a library inside your service with optional exactly-once.

Schema Registry & friends

Kafka stores raw bytes and doesn't care what they mean — but your producers and consumers must agree, even as data evolves over years. The **Schema Registry** is a separate service that stores versioned schemas (Avro, Protobuf, or JSON Schema), hands out schema IDs, and enforces **compatibility** rules so a new producer can't break old consumers.



FIG 7.3 The Schema Registry. Producers register a schema and get an ID, which is embedded (with a magic byte) ahead of the binary payload; consumers read the ID and fetch the matching schema (cached). Compatibility modes let schemas evolve over time without breaking either side — the registry refuses changes that would.

...and friends

Two more tools round out the platform. **ksqlDB** lets you express stream processing as SQL (`CREATE STREAM ... SELECT ...`), compiling to Kafka Streams under the hood — great for quick transformations and materialized views without writing Java. **MirrorMaker 2** replicates topics *between* Kafka clusters (for geo-replication, migration, or disaster recovery), preserving partitioning and translating consumer offsets across clusters.

HOW THE ECOSYSTEM FITS TOGETHER

Connect gets data in and out without code; the **Schema Registry** keeps that data well-typed and evolvable; **Streams** (or **ksqlDB**) processes it with state and exactly-once; and **MirrorMaker** moves it between clusters. Underneath, every one of them is just producing to and consuming from the same partitioned, replicated log you now understand in depth.

VIII

PART EIGHT

Mastery

You now hold every piece. Mastery is knowing how the pieces pull against each other — and reaching for the right knob under pressure. This part is the map of trade-offs, the pitfalls that bite real teams, and a glossary to make the vocabulary permanent.

CHAPTER 24 • PUTTING IT TOGETHER

The trade-off triangle & the field guide

Almost every Kafka tuning decision is a negotiation between four desires: **durability**, **throughput**, **low latency**, and **ordering**. You rarely maximize all of them at once — pushing one usually costs another. Knowing which knob moves which is the whole game.

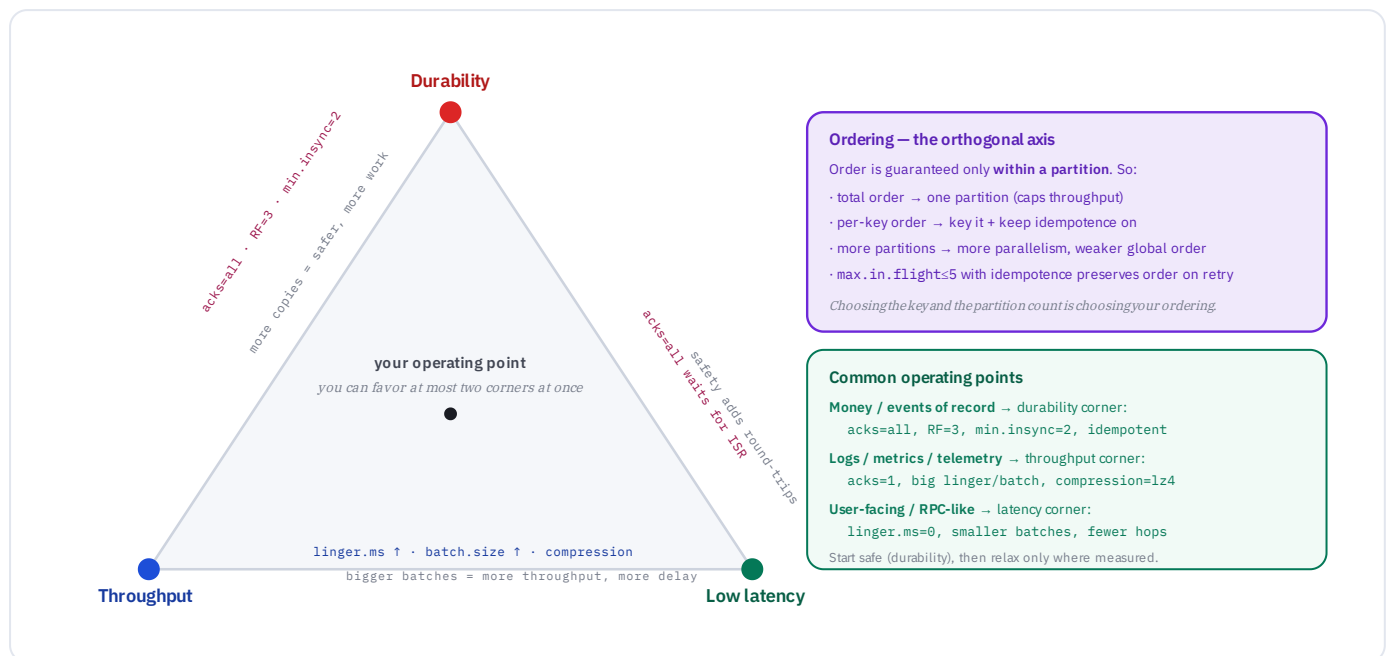


FIG 8.1 The trade-off triangle. Durability, throughput, and latency pull against one another; the configs on each edge are the levers that move you between corners. Ordering is a separate axis governed by keys and partition count. There is no universally "correct" tuning — only the right point for a given workload.

Common pitfalls — and how to avoid them

Pitfall	What happens	Fix
Too few partitions	Consumer parallelism is capped; you can't scale out reads.	Size partitions for peak consumer count up front; over-provision modestly.
Hot keys / skew	One partition gets most traffic; one consumer saturates while others idle.	Choose a higher-cardinality key, or add a salt/sub-key for the hot entity.
Poison pill	One un-deserializable/erroring record stalls a partition as the consumer retries forever.	Wrap processing in error handling; route bad records to a dead-letter topic.
Rebalance storms	Slow processing trips <code>max.poll.interval.ms</code> ; the consumer is evicted, triggering endless rebalances.	Do less per poll, raise the interval, use cooperative-sticky + static membership.
Unbounded lag	Producers outpace consumers; end-to-end delay grows without limit.	Monitor lag; add consumers ($\leq$ partitions) or speed up/parallelize processing.
Large messages	Multi-MB records blow past <code>max.message.bytes</code> and bloat memory.	Store the blob elsewhere (object store) and send a reference/claim-check.
Unclean leader election on	An out-of-sync replica becomes leader → silent data loss.	Keep <code>unclean.leader.election.enable=false</code> for important topics.
Auto-commit + slow work	Offsets commit on a timer before processing finishes → silent message loss on crash.	Use manual commits after processing, or design idempotent handlers.

The tuning cheat-sheet

● Maximize durability

- `acks=all, enable.idempotence=true`
- `replication.factor=3, min.insync.replicas=2`
- `unclean.leader.election.enable=false`
- Manual commit *after* processing; transactions for EOS.

● Maximize throughput

- Raise `batch.size` & `linger.ms` (e.g. 32–64 KB / 10–100 ms).
- `compression.type=lz4` or `zstd`.
- Enough partitions to spread load; consider `acks=1` if loss-tolerant.
- Give brokers lots of RAM (page cache); keep heap modest.

● Minimize latency

- `linger.ms=0`, smaller batches.
- Keep `fetch.min.bytes` low so consumers don't wait to fill.
- Fewer network hops; co-locate clients with brokers where possible.
- Accept the durability/latency tension: `acks=all` still costs a round-trip.

● Preserve ordering

- Key records by the entity whose order matters.
- `enable.idempotence=true` (keeps order on retry, ≤ 5 in-flight).
- Total order only with a single partition (throughput cost).
- Avoid increasing partition count on keyed, order-sensitive topics.

KEY IDEA • THE MASTERY MOVE

When something is wrong, name the *desire* first — "I'm losing data," "I can't keep up," "events arrive out of order," "it's too slow" — and the relevant corner of the triangle tells you which small set of knobs to reach for. Start from the safe defaults (durability), measure, and relax deliberately. That discipline, more than memorizing configs, is what mastery looks like.

Glossary — every term, one place

The vocabulary of Kafka, distilled. Each entry is the one-line mental model to anchor the term; flip back to the linked ideas in earlier chapters when you need the full picture.

Record / message / event

The atomic unit: a key, a value (both bytes), a timestamp, optional headers, plus an offset and partition assigned on append.

Key

Optional bytes that choose a record's partition (and thus its ordering group) and serve as the identity for compaction.

Topic

A named stream of records; logically one feed, physically split into partitions.

Partition

An independent, ordered, append-only log. The unit of ordering, parallelism, and replication.

Offset

A monotonically increasing integer identifying a record's position within a partition; starts at 0 per partition.

Broker

A single Kafka server that stores partitions, leads some, follows others, and serves clients.

Cluster

A set of cooperating brokers that together hold all topics and partitions.

Producer

A client that writes records, handling serialization, partitioning, batching, and retries.

Consumer

A client that reads records from assigned partitions via a poll loop and tracks progress with offsets.

Consumer group

A set of consumers sharing a `group.id`; partitions are divided so each is owned by exactly one member. Different groups read independently.

Group coordinator

A broker that manages a group's membership, assignments, and committed offsets.

Rebalance

Redistribution of partitions among a group's members when membership changes. Eager pauses everyone; cooperative moves only what must move.

Leader / follower

For each partition, one replica (leader) handles all traffic; others (followers) replicate it and can take over.

Replica / replication factor

A copy of a partition on a broker; the replication factor is how many copies exist.

ISR (in-sync replicas)

The replicas currently caught up enough to count toward durability and be eligible for leadership.

High watermark

The highest offset replicated to all in-sync replicas — the boundary consumers may read up to.

Log-end offset (LEO)

The offset where the next record will be appended on a given replica.

acks

Producer setting for how many acknowledgements to wait for: 0 (none), 1 (leader), all (in-sync replicas).

min.insync.replicas

The minimum ISR size required for an `acks=all` write to succeed; the durability floor.

Segment

A chunk of a partition's log stored as a file; only the newest (active) segment is open for writes.

Retention

The policy for discarding old data — by time/size (`delete`) or by collapsing to latest-per-key (`compact`).

Compaction / tombstone

Keeping only the newest value per key; a null-valued record (tombstone) marks a key for deletion.

Zero-copy

Sending data straight from the OS page cache to the network socket, bypassing the application — a key to Kafka's read throughput.

Controller

The component that maintains cluster metadata, detects broker failures, and drives partition leader elections.

KRaft

Kafka's built-in Raft-based metadata quorum that replaces ZooKeeper; metadata is itself a replicated log.

ZooKeeper

The external coordination service used by legacy Kafka for metadata and controller election; being retired.

Idempotent producer

A producer whose retries can't create duplicates, via a producer ID and per-partition sequence numbers.

Transaction

An atomic, all-or-nothing set of writes across partitions, coordinated by a transaction coordinator and made visible via control markers.

Delivery semantics

At-most-once (may lose), at-least-once (may duplicate), exactly-once (one effect) — determined by commit ordering and transactions.

read_committed

A consumer isolation level that hides records from uncommitted or aborted transactions.

Consumer lag

High watermark minus committed offset — how far a group is behind the latest data. The key health metric.

__consumer_offsets

The internal compacted topic where consumer groups store their committed offsets.

Kafka Connect

A framework for moving data between Kafka and external systems via configurable source and sink connectors.

Connector / task / converter / SMT

A connector defines integration; tasks parallelize it; converters (de)serialize; Single Message Transforms tweak records in flight.

Kafka Streams

A Java library for stateful stream processing, defined as a topology, with local state backed by changelog topics.

KStream / KTable

A stream of events (inserts) versus a table of latest-value-per-key (upserts) — the stream/table duality.

State store / changelog

Local storage (e.g. RocksDB) for streaming state, backed by a compacted Kafka topic for fault-tolerant recovery.

Schema Registry

A service storing versioned schemas (Avro/Protobuf/JSON) and enforcing compatibility; records carry a schema ID.

Compatibility mode

Rules (backward/forward/full/none) governing how schemas may evolve without breaking producers or consumers.

ksqlDB

A SQL layer over Kafka Streams for expressing stream processing and materialized views declaratively.

MirrorMaker 2

A tool for replicating topics between Kafka clusters, preserving partitioning and translating offsets.

Bootstrap servers

The initial broker addresses a client contacts to discover the rest of the cluster's metadata.

Partitioner

The producer component that maps a record (by key hash, round-robin, or custom logic) to a partition.

THE WHOLE PICTURE

Every capability in this guide — replication, consumer groups, transactions, Streams, Connect — is built on one idea you met on the very first page: **an append-only, partitioned, replicated log**. Producers append to it, brokers copy and store it, consumers read it at their own pace, and the entire ecosystem is just clever ways of writing to and reading from that log. Hold onto the log, and the rest of Kafka follows.

